UNIDADE 404: FUTURO NÃO ENCONTRADO — Game Design Document

GDD v0.4 · atualização de decisões e divisão de trabalho · 22/10/2026 · Engine: Godot 4.x · Jogo de game jam com um único input

Legenda: **[DECIDIDO]** = já definido pela equipe. **[PROPOSTA]** = sugestão a validar nos testes. Tudo marcado como proposta pode ser mudado sem quebrar o resto.

Registro de alterações — v0.4 (22/10/2026)

- **Nova gramática:** APARAFUSAR = clique longo; DESAPARAFUSAR = clique rápido + clique longo (sequenciais).
- **Plataforma:** apenas Windows (.exe), sem export web.
- **Estado do projeto:** M0 a M3 implementados; a remoção existente precisa de ajuste; backlog de dez issues para três desenvolvedores (seção 16).
- **Arquitetura:** cenas isoladas e contratos de sinais, sem criar actions.gd desnecessário (seção 14).
- **T_MIN:** estimativa atualizada para o parafuso enferrujado (cerca de 3,35 s); valor final sujeito a testes.
- **Limpeza:** removidas da lista de decisões em aberto as que já foram decididas (idioma, grafia “Paraguai”, volta ao menu).

1. Visão geral

Pitch: você é um robô de linha de montagem. Uma esteira traz peças com pregos e parafusos para pregar, aparafusar e remover, tudo com um único botão do mouse. Uma barra de tempo só desce, e cada slot bem feito compra alguns segundos. Um erro, ou o tempo acabando, e você é desligado da tomada; outro robô assume o seu lugar, porque a fábrica nunca para. O futuro não será encontrado.

- **Nome: Unidade 404: Futuro Não Encontrado** [DECIDIDO]. Aparece no menu, na janela do jogo, no nome do arquivo exportado e na tela preta do game over (seção 10).
- **Gênero:** arcade de precisão e ritmo com pontuação, partidas curtas. [DECIDIDO]
- **Perspectiva:** primeira pessoa em 2D / pixel art. O jogador vê a esteira à frente e as mãos e a ferramenta do robô embaixo. [DECIDIDO]
- **Input:** somente o botão esquerdo do mouse. Sem mira, sem movimento do mouse, sem teclado. [DECIDIDO]
- **Tema da jam:** o próprio input único é o tema.

- **Estrutura:** menu (jogar, placar, sair) → cutscenes → calibração opcional → jogo. [DECIDIDO]
- **Sessão:** 2 a 5 minutos por robô, reinício imediato.
- **Plataforma:** Windows desktop (.exe), execução local. Não haverá exportação web. [DECIDIDO]
- **Resolução base:** 320×180 com escala inteira (4× = 1280×720). [PROPOSTA]

Pilares de design

1. **Legibilidade:** o jogador sempre entende o que o jogo está pedindo agora.
2. **Justiça:** toda morte deve parecer culpa do jogador, nunca do sistema de input.
3. **Poucos sistemas, muito feedback:** cada ação tem som, movimento e resposta visual.
4. **Escopo de jam:** jogável com quadrados cinzas antes de ser bonito.
5. **Fim inevitável:** o jogador sabe que vai ser desligado. O jogo é sobre quanto ele dura, e o ritmo aperta aos poucos. [DECIDIDO a ideia]

2. Lore e tom

A Fábrica nunca para. Os humanos que a administram erram o tempo todo e sempre têm uma justificativa. Robôs não têm esse direito: cada unidade é produzida em série, descartável e com prazo. Um único erro de montagem, ou um atraso, e o operador humano desliga a unidade da tomada e ativa a próxima.

O jogador é essa unidade. Quando ele é desligado, sabe que o robô seguinte não é o mesmo: o número de série muda, o nome muda e o visor muda de cor. Ele também sabe, desde o começo, que vai ser desligado: ninguém dura nessa fábrica. [DECIDIDO a ideia; detalhes são PROPOSTA]

Unidade 404: o nome do jogo também é um código de erro (“não encontrado”). Na lore, todas as unidades são do modelo 404, o modelo para o qual não existe futuro. [PROPOSTA]

- **Tom:** humor negro, industrial, minimalista. Sem diálogos falados; texto curto no visor (“ATIVADA”, “DESLIGADA”).
- **O outro robô:** um braço mecânico de outro robô entra por cima da tela para repor peças (seção 4). Ele só ajuda e não fala.

Cutscenes de abertura [PROPOSTA de texto, a equipe ajusta]

Telas curtas de texto sobre fundo escuro, com o texto surgindo letra por letra (funcionamento na seção 9). O marcador {NOME} é trocado pelo nome gerado para o robô (seção 10.3):

1. A FÁBRICA NUNCA PARA.

2. Humanos erram. Eles têm justificativas.
3. Robôs não têm.
4. UNIDADE 404: {NOME}. ATIVADA. NÃO ERRE. NÃO DEMORE.

3. Input: gestos, ferramentas e leitura do clique

O jogo usa só o botão esquerdo do mouse. O controle é feito com três gestos e com a **escolha da ferramenta** antes de cada ação.

3.1 Gestos [DECIDIDO]

- **Clique:** um toque curto.
- **Duplo clique rápido:** dois toques seguidos. Não é o duplo clique do sistema operacional: o jogo lê os eventos do mouse e mede o intervalo.
- **Segurar:** manter o botão apertado.

3.2 Escolha da ferramenta [DECIDIDO]

Antes de cada interação com um slot, o jogador seleciona a ferramenta:

- **1 clique:** martelo.
- **2 cliques rápidos:** chave de fenda (aparafusadeira).

Regras:

- A seleção vale para cada slot, **mesmo que seja a mesma ferramenta usada no slot anterior**. Serve como confirmação.
- **Não dá para voltar atrás.** Se a ferramenta escolhida não é a que o slot pede, o robô é desligado na hora. Na lore, um robô deveria reconhecer sempre a ferramenta certa.
- Ao concluir um slot, a ferramenta é guardada e a mão volta ao estado neutro.
- **Exceção:** depois de remover uma peça ruim, o jogador **não** seleciona a ferramenta de novo para instalar a peça nova. A ferramenta continua na mão (seções 4 e 5).

3.3 Ações de cada ferramenta [DECIDIDO, atualizado na v0.4]

- **Martelo:** 1 clique martela o prego. 2 cliques rápidos removem o prego torto (o martelo vira garra).
- **Chave de fenda, APARAFUSAR:** 1 clique longo (segurar até completar o tempo configurado).
- **Chave de fenda, DESAPARAFUSAR:** sequência de **1 clique rápido seguido de 1 clique longo** (segurar até completar). É uma sequência, não um duplo clique. O clique curto inicial prepara e inverte o movimento da chave, e o clique longo remove o parafuso.

- Depois que o braço auxiliar instala um parafuso novo no slot enferrujado, a ação final é só APARAFUSAR (1 clique longo), sem nova seleção da ferramenta.
- A seleção da chave continua sendo 2 cliques rápidos na fase SELECT_TOOL. A sequência clique rápido + clique longo pertence exclusivamente à fase de remoção.
- Qualquer gesto concluído incompatível com a fase é erro (seção 6), observadas as exceções de cancelamento e pausa da seção 3.4.

3.4 Regras de leitura [PROPOSTA, revisada na v0.4]

Valores iniciais, centralizados no autoload Config e sujeitos a playtest:

- **JANELA_DUPLO:** 0,28 s, o intervalo máximo entre dois cliques para valerem como duplo clique.
- **LIMIAR_SEGURAR:** 0,30 s. Apertado por mais tempo que isso, deixa de ser clique e vira “segurar”.
- **TEMPO_APARAFUSAR:** 0,90 s de pressionamento contínuo para concluir um aparafusar ou desaparafusar.
- **ATRASO_ANEL:** 0,10 s. O anel de progresso só aparece depois disso.
- **TRAVA_POS_ACAO:** 0,15 s. Ignora cliques logo depois de uma ação, para evitar erro acidental.
- **JANELA_CLIQUE_LONGO:** 0,75 s. Constante nova: intervalo máximo entre o clique rápido de preparo e o início do clique longo, na remoção do parafuso. [PROPOSTA, sujeita a testes de justiça do input]

Regras:

1. O ClickReader mede os eventos do botão esquerdo e diferencia clique, duplo clique e segurar. Não usa o duplo clique do sistema operacional.
2. **Seleção da ferramenta:** 1 clique confirmado = martelo; 2 cliques rápidos = chave. Segurar na fase SELECT_TOOL é ignorado no modo normal.
3. **Fases de instalação:** segurar até TEMPO_APARAFUSAR completa APARAFUSAR. Soltar antes não é erro: o progresso é cancelado e pode ser reiniciado.
4. **Remoção do parafuso enferrujado:** um estado intermediário **REMOVE_PREPARED** começa depois de um clique curto reconhecido. O clique longo seguinte conclui DESAPARAFUSAR. O clique curto sozinho não remove nada. Duplo clique ou clique longo sem preparo são ações incorretas, depois da classificação definitiva do gesto. Se a JANELA_CLIQUE_LONGO expirar, o preparo pode ser reiniciado sem desligar o robô. [PROPOSTA]
5. Um pressionamento longo que começa como o segundo da sequência não pode ser classificado como duplo clique. O ClickReader e o controlador precisam distinguir a duração do segundo pressionamento antes de finalizar o gesto.

6. Durante a substituição automática, o movimento da esteira e o desligamento, os eventos de mouse são ignorados e a barra de tempo não drena.
7. Segurar 1,5 s na calibração, quando nenhuma ferramenta está selecionada, pula a calibração (seção 9.4).

O ClickReader classifica eventos físicos; o InteractionController interpreta o gesto conforme a fase do slot (seção 14). A nova remoção precisa do estado intermediário e de testes, automatizados ou manuais.

Ritmo: para martelar, o jogador seleciona (1 clique), espera um instante e clica de novo. Se o segundo clique vier rápido demais, o jogo lê duplo clique e seleciona a chave. Isso faz parte do desafio, mas precisa de confirmação visual da ferramenta e de testes (seção 17).

3.5 Espera do clique e feedback [PROPOSTA, revisada na v0.4]

Um clique simples só é confirmado quando a janela termina. Para a espera não parecer atraso:

- **Seleção:** a ferramenta dá resposta inicial imediata (o indicador no visor pisca e toca um som curto), mas só confirma clique simples ao fim da janela de duplo clique. Sem segundo clique, o martelo aparece na mão. Com segundo clique, aparece a chave.
- **Martelo:** antecipa a animação de levantar e, conforme o gesto, martela ou usa a garra para remover o prego torto.
- **Chave de fenda:** mostra progresso circular enquanto o botão está pressionado. No DESAPARAFUSAR, o clique rápido inicial dá um feedback de preparo e inversão (som e indicador); só o clique longo seguinte gira a chave e remove o parafuso. Na instalação não há clique de preparo.
- **Anel de progresso:** fixo sobre a ferramenta, nunca segue o cursor. Soltar antes da conclusão cancela apenas a etapa de segurar.
- **Otimização futura, só se parecer lento:** esperar a janela apenas quando a fase atual aceita duplo clique.

3.6 Cursor [DECIDIDO a restrição]

- O cursor do mouse fica oculto durante o jogo e não há mira.
- Nenhuma tela exige apontar com o mouse. Todas as escolhas usam clique, duplo clique ou segurar.

3.7 Gramática dos menus [DECIDIDO]

- **Clique:** muda a opção destacada para a próxima, dando a volta ao chegar na última. Vale na hora, sem esperar a janela de duplo clique.

- **Segurar:** confirma a opção destacada. O anel enche sobre a opção até completar (MENU_SEGURAR = 0,8 s, valor inicial). Soltar antes cancela e zera o anel.
- Telas com opções mostram a instrução fixa “CLIQUE: MUDAR · SEGURAR: ESCOLHER”.
- Telas sem opções seguem a mesma ideia: na cutscene o clique completa o texto e passa para o próximo, sem pular nada; no placar, segurar volta ao menu, com anel.
- **Dois modos de leitura no ClickReader:** o modo **IMEDIATO** (menu, cutscene, placar) aceita o clique na hora; o modo **COM JANELA** (jogo e pergunta de calibração) espera a JANELA_DUPLO para distinguir clique de duplo clique.
- **Ponto a decidir:** a pergunta “CALIBRAR? 1 clique: sim · 2 cliques: não” usa uma gramática diferente da do menu. Uma alternativa coerente é usar a do menu (clique muda entre SIM e NÃO, segurar confirma). Ver seção 17.

4. A peça e seus slots [DECIDIDO, atualizado na v0.4]

Cada peça chega pela esteira com até 4 slots em fileira. O **slot ativo fica destacado (highlight)**. O jogador vê, olhando o slot, o que precisa fazer, sem texto. Há quatro tipos de slot (um asset de cada), com estas gramáticas completas:

1. **Prego normal:** 1 clique seleciona o martelo → 1 clique martela.
2. **Parafuso normal:** 2 cliques rápidos selecionam a chave → 1 clique longo aparafusa.
3. **Prego torto:** 1 clique seleciona o martelo → 2 cliques rápidos removem o prego → **automático:** o braço auxiliar traz um prego novo → 1 clique martela, **sem nova seleção**.
4. **Parafuso enferrujado:** 2 cliques rápidos selecionam a chave → 1 clique rápido + 1 clique longo desaparafusam → **automático:** o braço auxiliar coloca um parafuso novo → 1 clique longo aparafusa, **sem nova seleção**.

A ferramenta precisa ser selecionada de novo a cada slot novo, mesmo que seja a mesma. A única exceção é a continuidade dentro de um mesmo slot, depois da substituição.

Braço do outro robô: depois de uma remoção, um braço mecânico entra por cima da tela, deixa a peça nova (prego ou parafuso) e sai. Durante essa animação os cliques são ignorados. O estado seguinte é o de instalação, com a mesma ferramenta ainda selecionada.

A variedade visual das placas não altera a lógica: cada peça é uma lista de slots.

5. Ciclo de uma peça [DECIDIDO, atualizado na v0.4]

1. A peça entra na estação pela esteira.
2. O primeiro slot ativo recebe destaque.

3. O jogador seleciona a ferramenta e executa as fases da seção 4; o braço auxiliar substitui automaticamente as peças defeituosas. A ferramenta se desloca até o slot em destaque quando age. [PROPOSTA]
4. Concluído um slot, o destaque avança.
5. Concluído o último slot, a peça sai e a próxima entra.
6. O tempo drena apenas quando o jogador pode agir e pausa durante as automações. Se a barra zerar, o robô é desligado (seção 6.2).

Exemplo com quatro slots

1. **Prego:** 1 clique (martelo) e 1 clique (martelar).
2. **Parafuso:** duplo clique (chave) e clique longo.
3. **Prego torto:** 1 clique (martelo), duplo clique (remover), braço auxiliar e 1 clique (martelar).
4. **Parafuso enferrujado:** duplo clique (chave), 1 clique rápido e 1 clique longo (desaparafusar), braço auxiliar e 1 clique longo (aparafusar).
5. **Peça concluída:** a esteira avança.

6. Erro, tempo e consequência [DECIDIDO]

6.1 Erro

- **Erro por ferramenta errada:** escolher a ferramenta incorreta desliga o robô no jogo normal (ex.: 2 cliques escolhendo a chave num prego).
- **Erro por gesto errado:** executar um gesto incompatível com a fase também desliga o robô. Exemplos: martelo selecionado e o jogador segura; chave selecionada e o jogador clica; 1 clique quando pedia remover; 2 cliques quando pedia martelar; **clicar duas vezes quando a fase de remoção da chave espera clique rápido + clique longo**.
- **Não é erro:** segurar durante a seleção da ferramenta (ignorado); soltar o botão antes de concluir um clique longo; clicar durante ações automáticas; deixar expirar a preparação para desaparafusar, se a regra de cancelamento sem punição da seção 3.4 for adotada. [PROPOSTA]
- **Na calibração,** os erros mostram feedback e reiniciam o slot. No jogo normal, encerram a vida do robô.

6.2 Barra de tempo [DECIDIDO a ideia; números PROPOSTA]

A barra de tempo (no visor, com o rótulo “VIDA ÚTIL” ou “TEMPO”) fica **visível o tempo todo**. Ela só desce, 1 segundo por segundo de jogo, e **sobe um pouco a cada slot concluído**. Se chega a zero, o robô é desligado (seção 10). O jogador sabe que vai ser

desligado cedo ou tarde; a barra existe para apertar o ritmo e impedir que ele fique parado.

Funcionamento

- Começa com BARRA_INICIAL = 8 s e tem teto BARRA_MAX = 10 s. O que passa do teto é perdido, então não dá para acumular tempo.
- Cada slot concluído soma TEMPO_CONCEDIDO (abaixo).
- **A barra pausa** durante as sequências automáticas (braço do outro robô, movimento da esteira, desligamento), porque o jogador não controla esses momentos.
- Abaixo de 25% a barra pisca e toca um som grave curto, para dar chance de reação.
- Na calibração a barra aparece parada: não desce, não mata, e sobe a cada slot, só para o jogador entender o que ela significa.
- A barra é um saldo: ser rápido num slot compensa a lentidão no seguinte.

Como o tempo diminui com o progresso

TEMPO_CONCEDIDO = T_MIN do slot × MARGEM do nível. A margem cai a cada nível, então o mesmo slot rende menos tempo quanto mais peças o jogador já completou.

- MARGEM por nível (valores iniciais): nível 1 = 2,5 · nível 2 = 2,2 · nível 3 = 1,9 · nível 4 = 1,7 · nível 5 = 1,5 · nível 6 (teto) = 1,3.
- Depois do teto a margem continua caindo devagar (0,01 por peça) até um **piso duro de 1,10**. Nesse ponto ninguém sustenta o ritmo, e é isso que torna o fim inevitável. [PROPOSTA]

Como o limite humano é respeitado

O tempo concedido nunca fica abaixo do que uma pessoa precisa para executar a tarefa sem hesitar. Por isso:

1. **T_MIN é calculado, não chutado.** É a soma do tempo de reação com as durações mecânicas dos gestos do slot (janela de duplo clique, tempo de segurar, trava pós-ação). Se as constantes da seção 3.4 mudarem, T_MIN muda junto.
2. **Tempos de reação fixos:** REACAO_ESCOLHA = 0,45 s para um slot novo (olhar, reconhecer o tipo e decidir) e REACAO_SIMPLES = 0,25 s quando a ação já é conhecida (por exemplo, logo depois de o braço deixar a peça). São valores típicos de reação humana a estímulos visuais e **precisam ser validados em playtest**.
3. **Margem mínima:** a margem nunca cai abaixo do piso duro de 1,10, e no teto de dificuldade fica em 1,30, exigente mas possível para um bom jogador.
4. **Níveis iniciais generosos** (margem 2,5), para quem ainda aprende os gestos não ser punido pelo relógio.
5. **Automáticos fora da conta:** o braço e a esteira não consomem a barra.

6. **Ajuste com dados:** em modo de teste o jogo registra, por tipo de slot, quanto cada testador levou. A margem do teto é ajustada para que um jogador mediano morra no teto e um jogador muito bom consiga sustentá-lo por um tempo.

Exemplos de T_MIN com as constantes iniciais (o código calcula, ninguém precisa decorar). Premissas: toque de 0,10 s e intervalo humano de 0,12 s entre os dois cliques de um duplo clique.

- **Prego:** reação 0,45 + selecionar martelo 0,38 + martelar 0,38 + trava 0,15 ≈ **1,4 s**.
- **Parafuso:** 0,45 + selecionar chave 0,32 + segurar 0,90 + trava 0,15 ≈ **1,8 s**.
- **Prego torto:** 0,45 + selecionar 0,38 + remover 0,32 + reação 0,25 + martelar 0,38 + trava 0,15 ≈ **1,9 s** (sem contar o braço).
- **Parafuso enferrujado:** reação 0,45 + seleção 0,32 + clique de preparo 0,38 + segurar 0,90 + reação após a substituição 0,25 + segurar 0,90 + trava 0,15 ≈ **3,35 s** (sem contar o braço). [PROPOSTA; recalcular conforme a janela de preparo e os tempos observados]

No nível 1 (margem 2,5) um prego rende 3,4 s de barra. No teto (margem 1,3) rende 1,8 s.

6.3 Consequência

- **No jogo normal:** erro ou barra zerada desliga o robô na hora (seção 10).
- **Na calibração:** o erro mostra feedback (flash vermelho, buzzer curto, “AÇÃO INCORRETA”), o slot volta ao início e o jogador refaz. Nada avança e nada mata.

7. Dificuldade e progressão [PROPOSTA]

O nível sobe a cada 5 peças concluídas e **tem teto**. Valores iniciais, para ajustar em teste:

- **Nível 1:** 1 slot, sem peça ruim, margem 2,5.
- **Nível 2:** 1 a 2 slots, sem peça ruim, margem 2,2.
- **Nível 3:** 2 slots, 15% de chance de peça ruim por slot, margem 1,9.
- **Nível 4:** 2 a 3 slots, 25%, margem 1,7.
- **Nível 5:** 3 a 4 slots, 30%, margem 1,5.
- **Nível 6 (teto):** 4 slots, 35%, margem 1,3.

“Peça ruim” é o slot de prego torto ou parafuso enferrujado, que exige mais passos (seleção, remoção, braço, instalação). Depois do teto o jogo continua com peças aleatórias dentro do teto, enquanto a margem de tempo segue caindo devagar até o piso duro (seção 6.2). Um gerador de peças (item_factory) recebe o nível e devolve a peça. O modelo de tempo (time_model) recebe o nível e o tipo de slot e devolve o tempo concedido. Esses dois são os únicos lugares onde a dificuldade vive.

8. Pontuação [PROPOSTA]

- 10 pontos por slot concluído.
- 50 pontos por peça concluída.
- Não há bônus de velocidade: a pressão de ritmo agora vem da barra de tempo.
- O **tempo de vida** do robô (em segundos, contado só no modo normal) é guardado junto com a pontuação.
- No fim de cada vida, pontuação, tempo e nome vão para o placar se entrarem entre os 3 melhores (seção 9.2). O recorde mostrado no visor é o do primeiro colocado.

9. Menu, placar, abertura e calibração

9.1 Menu principal [DECIDIDO]

Tela inicial com o nome do jogo e três opções: **JOGAR**, **PLACAR** e **SAIR**.

- **Clique** muda a opção destacada. **Segurar** escolhe, com o anel enchendo sobre a opção até completar (seção 3.7).
- A opção JOGAR começa destacada.
- Instrução fixa embaixo: “CLIQUE: MUDAR · SEGURAR: ESCOLHER”.
- **SAIR** fecha o jogo no computador.
- O menu usa o modo de leitura IMEDIATO.

9.2 Placar [DECIDIDO a ideia; detalhes PROPOSTA]

- Acessível pelo menu. Mostra os **3 melhores jogadores locais**, em 3 colunas: **NOME**, **TEMPO** e **PONTUAÇÃO**.
- Cada linha é um robô (uma “vida”): o nome que ele recebeu, quanto tempo viveu (mm:ss, só no modo normal) e a pontuação que fez.
- **Ordenação fixa, calculada no código:** pontuação maior primeiro; empate decidido pelo tempo maior. Ao fim de cada vida o código compara com as 3 linhas, insere se entrar, reordena e descarta a quarta.
- Linhas vazias mostram “---”.
- Instrução na tela: “SEGURAR: VOLTAR”, com o anel enchendo enquanto o jogador segura. Ao completar, volta ao menu.
- **Salvamento local** em arquivo na pasta `user://` do Godot (ConfigFile ou JSON). Se o salvamento falhar, o jogo continua sem salvar.

9.3 Cutscenes de abertura [DECIDIDO]

Ao escolher JOGAR, o jogo gera o nome do primeiro robô e segue:

1. **Cutscenes:** telas de texto com a lore (seção 2). O texto surge letra por letra. **Um clique completa o texto** da tela e **o clique seguinte passa para a próxima**. Modo de leitura IMEDIATO.
2. **Sem pular:** a cutscene não pode ser pulada inteira; o jogador avança texto a texto com cliques. [DECIDIDO]
3. **Pergunta de calibração:** "CALIBRAR? 1 CLIQUE: SIM · 2 CLIQUES: NÃO". Modo de leitura COM JANELA.
4. Sim leva à calibração. Não leva direto ao jogo.

9.4 Calibração [DECIDIDO a ideia; detalhes PROPOSTA]

- Fácil, com todas as possibilidades de peça, e **sem game over**. O erro mostra feedback e o jogador refaz o slot (seção 6.3). A barra de tempo aparece parada (seção 6.2).
- **Sequência (cerca de 8 peças):** 1) um prego; 2) um parafuso; 3) prego torto, com a chegada do braço; 4) parafuso enferrujado, com a chegada do braço; 5) peça mista de 3 slots; 6) peça mista de 4 slots com todas as possibilidades.
- **Dicas:** o visor mostra o gesto pedido ("1 CLIQUE", "2 CLIQUES", "SEGURAR"). As dicas só existem na calibração.
- **Sempre dá para pular,** de três formas:
  - responder "não" na pergunta;
  - **inatividade:** 5 segundos sem agir mostram uma mensagem pedindo ação (ex.: "UNIDADE INATIVA. AGIR."), e mais 5 segundos parado pulam a calibração;
  - **segurar o clique por 1,5 s** (com anel) quando nenhuma ferramenta está na mão. Nessa fase segurar não é um gesto válido, então não conflita com aparafusar. O visor mostra "SEGURAR: PULAR" nesses momentos.

Depois de perder: se o jogador clicar na tela preta do game over, o jogo volta direto ao modo normal com o novo robô (novo nome, nova cor), sem cutscenes e sem a pergunta de calibração. Se não reagir em 5 s, volta ao menu inicial (seção 10.1).

10. Game over, nova unidade e nome do robô

10.1 Sequência de desligamento [DECIDIDO]

É a opção B: curta (cerca de 2 a 3 segundos) e **obrigatória, sem poder ser pulada**.

1. Ao errar ou zerar a barra de tempo, a esteira congela e os cliques são ignorados.
2. O LED vermelho da parede começa a piscar e um alarme toca.

3. A tela desliga: a imagem encolhe até virar uma linha e depois um ponto, com som de power-down.
4. **Tela preta, em estilo de tela de erro.** [PROPOSTA de texto]
    - Primeira linha: “UNIDADE 404: FUTURO NÃO ENCONTRADO” (o nome do jogo).
    - **Causa:** “AÇÃO INCORRETA” ou “TEMPO ESGOTADO”, para o jogador entender por que morreu.
    - O nome do robô, a pontuação e o tempo de vida; “NOVO RECORDE” se entrou no placar.
    - Depois de cerca de 1,5 s (cliques ignorados nesse tempo, para evitar clique acidental) aparece a instrução “CLIQUE PARA JOGAR DE NOVO”. Um clique liga o novo robô (seção 10.2). Se o jogador não reagir em 5 segundos, o jogo volta ao menu inicial, para que uma pessoa ausente não fique reiniciando o jogo para sempre. [DECIDIDO a regra; o bloqueio de 1,5 s é PROPOSTA]

10.2 Nova unidade [DECIDIDO a ideia; detalhes PROPOSTA]

- Cada unidade tem **nome**, **número de série** (Nº 001, 002, 003...) e **cor**.
- Saindo do preto, o visor liga (efeito inverso do desligar) **já como outro robô, com a UI em outra cor**, e mostra “UNIDADE 404: {NOME} ATIVADA”.
- **Cor:** paleta fixa de 6 cores que cicla pelo número de série (nunca repete a do robô anterior). É aplicada com a propriedade modulate do Godot na parte tingível da moldura do visor e nos braços.
- **Regras da paleta:** cores bem diferentes entre si e legíveis sobre o fundo escuro; **sem vermelho** (reservado ao LED e aos erros); arte-base em cinza claro, porque o modulate multiplica a cor. O tamanho da paleta não multiplica assets.
- O jogo recomeça no nível 1, com a barra no valor inicial e pontuação zero. O LED da parede volta ao normal.

10.3 Nome do robô [DECIDIDO a ideia; detalhes PROPOSTA]

- **Arquivo da equipe:** duas listas de 20 itens. A **lista 1** tem substantivos do universo da fábrica e da tecnologia (ex.: Sucata, Operário, Chassi, Unidade, Androide), cada um com o gênero (“m” ou “f”). A **lista 2** tem adjetivos e expressões de humor (ex.: Cansado, em Greve, Sem Memória, da Shopee), cada uma com a forma masculina e a feminina. O nome é sempre uma **dupla**: um item da lista 1 mais um da lista 2, separados por espaço (400 combinações).
- **Concordância de gênero:** o gênero do substantivo decide qual forma do complemento entra (“Operário Cansado”, “Unidade Cansada”). A função `gerar_nome_robo()` do arquivo já faz isso.

- Gerado a cada nova unidade (para a primeira, ao escolher JOGAR). Opcional: evitar repetir o nome do robô anterior e os últimos da sessão (ex.: 5).
- Guardado em `GameState.robot_name`, com o gênero em `GameState.robot_gender`. O sinal `robot_changed(name, gender, serial, color_index)` avisa quem precisa reagir.
- **Fácil de referenciar:** todo texto do jogo pode usar os marcadores {NOME} e {SERIE}. Uma função utilitária (text_utils) troca os marcadores na hora de mostrar o texto. Vale para cutscene, mensagens do visor, tela preta do game over e placar.
- **Textos fixos:** escrever as frases do jogo com sujeito feminino fixo (“UNIDADE 404 ATIVADA”, “UNIDADE DESLIGADA”, “UNIDADE INATIVA”), para não depender do gênero do nome.
- **Nomes que repetem o texto do jogo:** “Unidade Desligada” ou “Unidade Defeituosa” repetem o texto do jogo (ex.: “UNIDADE 404: Unidade Desligada”). Decidido: manter a palavra “Unidade” na lista, como humor intencional.
- **Onde aparece:** visor (canto fixo), cutscene (tela 4), tela preta do game over, placar e mensagem de ativação. Não há diálogos previstos.
- **Cor do texto do nome:** neutra (tom claro de fósforo do visor) em todas as unidades. Mantém a legibilidade e evita um asset de fonte por cor. A cor do robô fica na moldura e nos braços. Testar o nome colorido depois, se sobrar tempo.
- **Fonte:** pixel com **suporte obrigatório a acentos** (á, ã, é, í, ó, ô, ú), porque as listas têm Operário, Robôzinho, Autômato, Taubaté, Memória, Prejuízo e Pressão. A fonte será escolhida depois, com visual pixel art coerente. O nome mais longo tem 23 caracteres (“Estagiário Desempregado”), e o espaço do visor precisa comportar isso.
- **Revisão do conteúdo:** a lista usa a grafia “Paraguai”. As piadas regionais (Taubaté, Shopee, Paraguai) ficam, porque o jogo é só para o público brasileiro.

11. Direção visual

- **Pixel art 320×180**, paleta curta, estilo industrial. A UI será feita no Photoshop e entregue em PNG.
- **UI diegética:** toda a interface é o visor do robô. Uma moldura (cantos, vinheta) cobre a tela e o HUD aparece nela. [PROPOSTA]
- **HUD do visor:** nome do robô (canto superior esquerdo, discreto, com o Nº de série), pontuação e recorde (canto superior direito), **barra de tempo** (topo, centro, sempre visível), indicador de ferramenta (martelo e chave, com a selecionada acesa).
- **Barra de tempo:** preenchimento em cor neutra que muda para âmbar abaixo de 25% e pisca. Não usa vermelho nem as cores da paleta dos robôs. A cada slot concluído ela dá um pulso rápido ao subir. [PROPOSTA]

- **Identidade por cor:** a moldura do visor e os braços do robô são tingidos pela cor da unidade. Cada peça de arte que recebe cor é entregue em **duas camadas**: uma tingível (cinza claro) e outra com os detalhes de cor fixa. Assim a quantidade de cores da paleta não multiplica os assets.
- **Layout do jogo:** topo = HUD; meio = esteira com a peça e o slot em destaque; embaixo = mãos e ferramenta; fundo = parede da fábrica com o LED de alarme.
- **Ferramenta visível na mão:** [DECIDIDO] martelo, chave de fenda e martelo-garra. A ferramenta se desloca até o slot em destaque quando age. [PROPOSTA]
- **Braço do outro robô:** garra mecânica que entra por cima, em cor diferente da do jogador para não confundir.
- **Menu e placar:** usam o mesmo visor (moldura e fundo da fábrica escurecido). O menu mostra o nome do jogo no topo e as três opções empilhadas, com a destacada em moldura ou brilho. O placar é uma tabela de 3 linhas e 3 colunas (NOME, TEMPO, PONTUAÇÃO).

Animações necessárias (podem começar com um sprite e rotação ou posição, sem frames desenhados):

- Seleção: o indicador de ferramenta pisca e a ferramenta aparece na mão.
- Martelo: levantar (preparação), bater, virar garra, puxar prego.
- Chave: clique curto de preparo para remoção, giro reverso no clique longo e giro normal na instalação.
- Slot: destaque pulsando, concluído, vazio e substituído.
- Braço do outro robô: entrar por cima com a peça, soltar no slot, sair.
- Esteira: andar e parar.
- Barra de tempo: pulso ao subir, piscar quando baixa.
- Menu: opção destacada trocando, anel enchendo sobre a opção.
- Efeitos: faíscas (partículas), tremor de tela leve, flash vermelho de erro, LED piscando, tela desligando e ligando.

12. Áudio

- Seleção de ferramenta: um som curto para o martelo e outro para a chave.
- Martelada, parafuso girando (curto e repetível), puxão de prego, parafuso sendo removido.
- Peça concluída (ding), erro (buzzer curto), alarme de desligamento, power-down e boot da nova unidade.
- Barra de tempo: tique suave ao ganhar tempo e som grave curto quando está baixa.

- Menu e placar: som curto ao mudar de opção, som de confirmação quando o anel completa.
- Braço do outro robô entrando e soltando a peça.
- Cutscenes: som curto de “digitação” enquanto o texto aparece.
- Motor da esteira (loop baixo). Música é opcional e a última prioridade.
- A reprodução de áudio será validada na versão Windows exportada.

13. Lista de assets

Prioridades: **P0** = necessário para ficar jogável (pode ser placeholder); **P1** = faz o jogo parecer pronto; **P2** = só se sobrar tempo. Origem: arte autoral em pixel art (Aseprite, LibreSprite ou Piskel) ou packs gratuitos (ex.: Kenney, OpenGameArt, itch.io). Conferir a licença de cada um e registrar a fonte.

Arte

- P0: fundo da fábrica (parede e piso), 320×180.
- P0: esteira (textura repetível que rola por código, ou 2 a 4 frames).
- P0: placa base da peça com até 4 slots (1 variante).
- P0: **os 4 tipos de slot:** prego esperando martelada, parafuso esperando aperto, prego torto ou quebrado, parafuso enferrujado.
- P0: estados de slot concluído e vazio: prego martelado, parafuso apertado, furo vazio.
- P0: destaque do slot ativo (moldura ou brilho).
- P0: braço do outro robô (garra) segurando um prego e segurando um parafuso, com animação de entrada por cima.
- P0: mão e braço do robô em cinza claro (camada tingível).
- P0: martelo (repouso, levantado, batida, garra) e chave de fenda (parada, girando). Podem começar como um sprite com rotação.
- P0: anel de progresso (textura circular ou TextureProgressBar).
- P0: moldura do visor, 320×180, centro transparente, em duas camadas (tingível e detalhes fixos).
- P0: **barra de tempo:** moldura e preenchimento.
- P0: ícones do indicador de ferramenta (martelo e chave, apagado e aceso).
- P0: LED vermelho da parede (apagado e aceso).
- P0: **fonte pixel livre com acentos** (cutscenes, menu, placar, nome do robô).
- P0: **menu:** nome do jogo em texto, marcação da opção destacada.
- P1: arte do logotipo “UNIDADE 404: FUTURO NÃO ENCONTRADO” e moldura da tabela do placar.

- P1: ícone do jogo (janela e arquivo).
- P1: faíscas (quadradinho branco com partículas, sem arte).
- P2: variações de placas e ilustrações curtas para as cutscenes.

A paleta de cores dos robôs é uma lista de valores no código, não um asset.

Áudio: tudo da seção 12, gerado com ferramenta de efeitos retrô (ex.: jsfxr) ou de bancos livres (ex.: Freesound, Kenney), conferindo a licença.

14. Estrutura no Godot (4.x)

O nome do projeto e da janela é “Unidade 404: Futuro Não Encontrado” (Project Settings > Application > Config > Name). Para o **arquivo exportado**, usar um nome sem dois-pontos e sem acentos (ex.: `unidade404`), porque o Windows não aceita “:” em nomes de arquivo. A cena inicial do projeto é `menu.tscn`.

```
res://
├── scenes/
│   ├── menu.tscn            (cena inicial do projeto)
│   ├── scoreboard.tscn
│   ├── intro.tscn
│   ├── game.tscn
│   ├── conveyor.tscn
│   ├── item.tscn
│   ├── slot_visual.tscn
│   ├── tools_visual.tscn
│   ├── helper_arm.tscn
│   ├── hands.tscn
│   ├── visor_hud.tscn
│   └── shutdown_fx.tscn
├── scripts/
│   ├── autoload/ (config.gd, game_state.gd)
│   ├── click_reader.gd
│   ├── interaction_controller.gd   (enums de ferramentas, slots e fases)
│   ├── slot.gd              (futuro, opcional)
│   ├── item.gd
│   ├── item_factory.gd
│   ├── time_model.gd
│   ├── run_manager.gd       (proposto)
│   ├── time_bar.gd
│   ├── conveyor.gd
│   ├── helper_arm.gd
│   ├── hands.gd
│   ├── menu.gd
│   ├── scoreboard.gd
```

```
|   ├── scoreboard_data.gd
|   ├── names_data.gd        (as 2 listas da equipe)
|   ├── name_generator.gd
|   ├── text_utils.gd
|   ├── intro.gd
|   ├── game.gd
|   ├── visor_hud.gd
|   └── shutdown_fx.gd
├── art/ (sprites, ui, fx)
├── audio/ (sfx, music)
└── fonts/
```

Árvore da cena do jogo (game.tscn)

- Game (Node2D, game.gd, implementação atual)
  - ClickReader (Node)
  - InteractionController (Node)
  - RunManager (Node, run_manager.gd, proposto)
  - TimeBar (Node, time_bar.gd)
  - World (Node2D): Background, WallLed, Conveyor (conveyor.tscn, com a peça atual), HelperArm (helper_arm.tscn)
  - Hands (hands.tscn): ToolSprite, AnimationPlayer, ProgressRing
  - VisorHud (CanvasLayer): Frame, nome e série, pontos e recorde, barra de tempo, indicador de ferramenta, HintLabel
  - Fx (CanvasLayer): FlashRect de erro, ShutdownFx
  - Audio (Node): AudioStreamPlayers
  - Intro (intro.tscn): cutscenes e pergunta de calibração, ativa só no começo

Responsabilidades (cada pedaço faz uma coisa só)

Os enums de ferramentas, slots e fases estão hoje em `interaction_controller.gd`. Não criar um `actions.gd` sem necessidade concreta.

- **click_reader.gd:** lê só InputEventMouseButton do botão esquerdo e ignora movimento do mouse. Mede tempos e produz gestos. Tem dois modos: IMEDIATO (menu, cutscene, placar) e COM JANELA (jogo e pergunta de calibração). **Não sabe nada de ferramentas, peças ou menus.**
- **interaction_controller.gd:** guarda a ferramenta na mão e a fase atual do slot ativo. Recebe um gesto e decide: seleção correta, ação correta ou erro. É onde vivem as regras das seções 3 e 6.1.
- **slot.gd e item.gd:** o slot tem tipo e lista de fases; a peça tem a lista de slots e o slot ativo.

- **item_factory.gd:** dado o nível, gera uma peça.
- **time_model.gd:** funções puras que calculam T_MIN de cada tipo de slot a partir das constantes e o TEMPO_CONCEDIDO por nível (seção 6.2).
- **run_manager.gd (proposto):** guarda os números da vida atual (pontuação, nível, saldo de tempo, tempo de vida), soma tempo ao concluir um slot, pausa nas automações, emite `time_expired` e encerra a vida com `finish_run`. A lógica de duração da partida fica aqui, e não em `game.gd`, para evitar conflitos entre branches.
- **time_bar.gd:** o saldo de tempo, a drenagem em tempo real e os sinais `time_changed(ratio)`, `time_low` e `time_over`. Pode ser absorvido pelo `run_manager.gd`; decisão de quem implementa (Dev 3).
- **conveyor.gd:** rola a esteira, para na estação, avança e troca a peça.
- **helper_arm.gd:** animação do braço do outro robô; avisa quando termina a substituição.
- **hands.gd:** animações e sons das ferramentas, reagindo aos sinais. Controla o anel e o deslocamento até o slot.
- **menu.gd:** opções, destaque, anel de confirmação e troca de cena.
- **scoreboard_data.gd:** carrega e salva o placar em arquivo, insere uma vida nova, ordena (pontuação, depois tempo) e corta em 3 linhas. **scoreboard.gd** só exibe esses dados.
- **names_data.gd e name_generator.gd:** o arquivo da equipe traz as duas listas e a função `gerar_nome_robo()`, que sorteia o substantivo e escolhe o complemento na forma concordante (masculino ou feminino). Pode ficar como um único arquivo (autoload NameGenerator) ou ser separado em dois. Evitar repetir os últimos nomes é opcional.
- **text_utils.gd:** troca {NOME} e {SERIE} em qualquer texto.
- **intro.gd:** cutscenes de texto e a pergunta de calibração.
- **game.gd:** coordena o ClickReader e o InteractionController e integra a esteira e os componentes visuais.
- **visor_hud.gd:** atualiza textos, barra de tempo, indicador de ferramenta e cor do visor.
- **shutdown_fx.gd:** sequência de desligamento (opção B), tela preta e a de religar.
- **Autoload Config:** constantes das seções 3.4 e 6.2, paleta de cores dos robôs e parâmetros de dificuldade.
- **Autoload GameState:** nome, gênero, série e cor do robô atual, e a função `new_robot()` que gera tudo de uma vez. [PROPOSTA: pontuação, nível e tempo de vida da vida atual ficam no RunManager, para não haver dois donos do mesmo número.]

Sinais (como as partes conversam)

- ClickReader: `click`, `double_click`, `hold_started`, `hold_progress(ratio)`, `hold_completed`, `hold_released`.

- InteractionController (já implementado): `tool_selected(tool)`, `step_completed`, `slot_completed`, `replacement_requested` e `wrong_action(reason)`. A nova sequência de remoção exige um estado adicional de preparação.
- RunManager (proposto): `time_expired`.
- TimeBar: `time_changed(ratio)`, `time_low`, `time_over`.
- Item: `slot_changed`, `item_completed`.
- HelperArm: `replacement_finished`.
- Conveyor: `movement_finished`.
- GameState: `robot_changed(name, gender, serial, color_index)`.
- Game: `state_changed(state)`. [proposto]

Fluxo de uma ação: o ClickReader produz o gesto → o InteractionController compara com a ferramenta na mão e a fase do slot → se estiver certo, avança a fase (ou conclui o slot, ou pede a substituição ao HelperArm) → ao concluir o slot, o `game.gd` avisa o RunManager, que pede o tempo ao time_model e soma na barra → Item, VisorHud, Hands e Audio reagem aos sinais. Se estiver errado, ou se o tempo zerar, o RunManager encerra a vida e o `game.gd` escolhe entre o feedback da calibração e o desligamento.

Dados de peça (recomendado): definir o slot e a peça como Resource do Godot facilita ajustar e testar variações sem mexer em código.

Salvamento: o placar vai para um arquivo em `user://`. Isso funciona na versão Windows.

15. Estados do jogo

1. **MENU:** JOGAR, PLACAR, SAIR.
2. **SCOREBOARD:** tabela dos 3 melhores; segurar volta ao MENU.
3. **INTRO:** cutscenes de texto (um clique completa o texto, o seguinte passa para a próxima; sem pular).
4. **ASK_CALIBRATION:** “1 clique: sim · 2 cliques: não”.
5. **CALIBRATION:** peças de treino, erro não mata, barra de tempo parada, temporizador de inatividade de 5 s + 5 s.
6. **PLAYING:** nível, peças, pontuação, barra de tempo correndo.
7. **SHUTDOWN:** sequência B obrigatória e tela preta com a instrução “CLIQUE PARA JOGAR DE NOVO”. Um clique leva ao REBOOT; 5 s sem clique voltam ao MENU.
8. **REBOOT:** sai do preto com a nova unidade e a UI em outra cor, mostra “UNIDADE 404: {NOME} ATIVADA” e volta ao PLAYING no nível 1.

Subestados dentro de PLAYING, por slot: **SELECT_TOOL → ACT → (AUTO_REPLACE → ACT_NO_SELECT)** → próximo slot ou fim da peça. A barra de tempo pausa em AUTO_REPLACE, no movimento da esteira e em SHUTDOWN.

16. Plano de produção — equipe de três desenvolvedores [ATUALIZADO]

Situação em 22/10/2026: M0 a M3 estão concluídos e testados localmente no Godot 4. O menu inicial, as cutscenes, os três gestos, a seleção de ferramenta e os quatro slots estão operacionais, numa interface temporária. O sistema de erro ainda reinicia slots para facilitar os testes.

Atenção: a remoção do parafuso enferrujado no protótipo ainda usa só clique longo e **deve ser alterada** para clique rápido seguido de clique longo. O M3 precisa dessa adequação antes de congelar a jogabilidade.

- **M0:** estrutura Godot 4, resolução e repositório. Concluído.
- **M1:** ClickReader com clique, duplo clique e segurar. Concluído; falta expandir o reconhecimento contextual para a nova remoção.
- **M2:** seleção de ferramentas e ações em slot. Concluído, exceto a nova gramática de remoção.
- **M3:** quatro tipos de slot, fases e progressão de teste. Concluído, exceto a nova gramática de remoção.
- **M4:** animação da esteira, troca de peças e braço auxiliar reutilizável.
- **M5:** visuais das ferramentas, sprites, HUD, progressão circular e feedback.
- **M6:** desligamento definitivo e reinício com novo robô.
- **M7:** gerador de peças, pontuação, tempo e dificuldade. **Aqui o jogo já é jogável.**
- **M8:** placar local persistente e aperfeiçoamento do menu já criado.
- **M9:** completar a pergunta de calibração e o treino; a abertura já está funcional.
- **M10:** áudio, arte, LED e alarme, desligamento completo e ajustes finais.
- **M11:** testes integrados, export Windows (.exe) e submissão.

16.1 Divisão para desenvolvimento em paralelo

- **Dev 1, Gameplay e integração:** propriedade principal de `game.gd` e `game.tscn`. Cria `conveyor.gd` e `conveyor.tscn`; cuida da progressão de slots e peças e do bloqueio de entrada durante animações; integra as cenas visuais e os sistemas de partida entregues pelos outros; adequa o InteractionController à nova remoção, com teste do ClickReader.

- **Dev 2, Arte e animações:** entrega assets e cenas independentes (`slot_visual.tscn` , `tools_visual.tscn` , `helper_arm.tscn`) com os scripts correspondentes. Também efeitos visuais e áudio. Não altera `game.gd` nem `game.tscn` ; testa cada cena com F6.
- **Dev 3 (Mari), Sistemas de partida e interface:** tempo, pontuação, dificuldade, game over, reinicialização, placar, HUD e persistência (M6 a M9), aproveitando o menu e a introdução existentes. Cria scripts e cenas independentes, reaproveitando GameState e Config. Não edita `game.gd` nem `game.tscn` .

Propriedade única de arquivos compartilhados: mudanças em `project.godot` , autoloads, `game.gd` e `game.tscn` devem ser coordenadas e revisadas pelo responsável pela integração (Dev 1). Usar branches e pull requests separados.

16.2 Contratos de integração [PROPOSTA; congelar antes de desenvolver]

- **Conveyor:** `signal movement_finished` .
- **HelperArm:** `func replace_part() -> void` ; `signal replacement_finished` .
- **ToolsVisual:** `func show_tool(tool: int) -> void` ; `func play_action(gesture: String) -> void` .
- **HUD:** `func set_score(points: int) -> void` ; `func set_time(seconds: float) -> void` ; `func set_level(level: int) -> void` (acrescentado nesta revisão, porque o HUD também mostra o nível). [PROPOSTA]
- **RunManager:** `signal time_expired` ; `func add_points(amount: int) -> void` ; `func finish_run(reason: String) -> void` .
- **Interface existente do InteractionController:** `tool_selected(tool)` , `step_completed` , `slot_completed` , `replacement_requested` , `wrong_action(reason)` , `set_slot(kind)` , `replacement_finished()` .

Cada desenvolvedor testa seu componente isoladamente, com sinais e dados simulados. As assinaturas propostas são acordos de integração, não funcionalidades já implementadas.

16.3 Backlog de issues definido pela equipe

- **I01:** esteira animada [Dev 1, alta].
- **I02:** progressão entre peças [Dev 1, alta].
- **I03:** integração gameplay e visuais [Dev 1, alta].
- **I04:** sprites de peças e ferramentas [Dev 2, alta].
- **I05:** braço auxiliar [Dev 2, alta].
- **I06:** feedback visual e áudio [Dev 2, média].
- **I07:** tempo, pontuação e dificuldade [Dev 3, alta].

- **I08:** desligamento e reinicialização [Dev 3, alta].
- **I09:** HUD e placar persistente [Dev 3, alta].
- **I10:** integração final, testes e export Windows [equipe, alta].
- **I11:** calibração e treino [Dev 3, média].

Primeira onda paralela: I01, I04 e I07. Dividir o trabalho em pull requests pequenos e reservar uma fase final exclusiva para integração e testes.

Relação dos marcos do Dev 3 com as issues: I08 = M6; I07 = M7; I09 = M8 (placar e HUD). O M9 (pergunta de calibração e treino) ainda não tem issue própria. [PROPOSTA: criar a I11, Calibração, Dev 3, média]

16.4 Critérios mínimos de entrega

Menu funcional; introdução obrigatória; quatro slots com os gestos atualizados; esteira; substituição automática; tempo e pontuação; game over; reinício; placar; executável Windows funcionando fora do editor. Efeitos extras, música, partículas e variações de placas são itens cortáveis.

Ordem de corte se faltar tempo: partículas, música, logotipo em arte, ilustrações das cutscenes, variações de placa, tela preta de erro detalhada (voltar ao texto simples), calibração reduzida (menos peças).

Playtest: observar pessoas jogando sem explicar nada. Perguntar “morreu sem entender por quê?”. Registrar quanto cada pessoa leva por tipo de slot e ajustar JANELA_DUPLO, LIMIAR_SEGURAR, JANELA_CLIQUE_LONGO e as margens de tempo com esses dados.

17. Riscos e decisões em aberto

Riscos

- **Morte injusta por input:** a escolha da ferramenta depende de um duplo clique, e errar o intervalo desliga o robô. A barra de tempo aumenta a pressa e, com ela, a chance de errar o intervalo. Mitigação: JANELA_DUPLO generosa (testar de 0,25 a 0,35 s), margem de tempo alta nos primeiros níveis, indicador de ferramenta com feedback imediato e calibração bem ensinada.
- **Clique rápido demais depois de selecionar o martelo** vira duplo clique e seleciona a chave. Mitigação: som e indicador mostram qual ferramenta foi escolhida; ajustar a janela nos testes.
- **Limite humano:** se as margens de tempo forem baixas demais, o jogo vira injusto. Mitigação: T_MIN calculado com tempos de reação, piso duro de 1,10 e validação com dados de playtest (seção 6.2).

- **Calibração opcional:** quem a pula pode morrer aprendendo. Mitigação: calibração curta e pergunta convidativa.
- **Escopo de arte:** 4 tipos de slot, braço do outro robô, ferramentas, visor em duas camadas, menu e placar. Mitigação: placeholders até o M10 e tingimento por cor.
- **Atraso do clique simples:** mitigado pela animação de preparação; otimização contextual como plano B.
- **Export Windows (.exe):** testar o .exe fora do editor (fontes embutidas, caminhos e salvamento do placar).
- **Nome do arquivo:** "Unidade 404: Futuro Não Encontrado" tem dois-pontos e acentos, que dão problema em nomes de arquivo. Usar um nome simples no arquivo e o nome completo na janela e no menu.
- **Nomes aleatórios:** a repetição de "Unidade" (ex.: "Unidade Desligada") é humor intencional. Revisar as listas só para evitar combinações ofensivas.
- **Gesto de remoção do parafuso (clique rápido + clique longo):** o segundo pressionamento longo pode ser lido como duplo clique, e a janela entre os dois pode ser injusta. Mitigação: classificar pela duração do segundo pressionamento (seção 3.4), JANELA_CLIQUE_LONGO configurável, cancelamento do preparo sem punição e testes do ClickReader.
- **Conflitos entre branches:** vários desenvolvedores mexendo em `game.gd`, `game.tscn`, `project.godot` e autoloads. Mitigação: propriedade única desses arquivos (seção 16.1), cenas e scripts independentes e pull requests pequenos.

Decisões em aberto

- **O que é "Unidade 404" na lore** e como se relaciona ao número de série. Proposta deste documento: 404 é o modelo de todas as unidades, e o número de série (Nº 001, 002...) identifica cada uma.
- **Bloqueio de cliques na tela preta.** Proposta: 1,5 s sem aceitar cliques antes de mostrar "CLIQUE PARA JOGAR DE NOVO" (seção 10.1).
- **Gramática da pergunta de calibração:** manter "1 clique sim, 2 cliques não" ou usar a gramática do menu (clique muda, segurar confirma).
- **Quantidade e cores da paleta dos robôs** (proposta: 6, sem vermelho).
- **Cor do nome no visor:** neutra (proposta) ou na cor do robô.
- **Definição de "tempo" no placar** (proposta: duração da vida do robô) e critério de ordenação (proposta: pontuação, depois tempo).
- Valores finais dos limiares (seção 3.4) e das margens de tempo (seção 6.2).
- Quantidade máxima de slots por peça (proposta: 4).
- Roteiro final das cutscenes e texto da mensagem de inatividade.
- Música ou apenas efeitos.

- **Janela de preparo da remoção:** valor de JANELA_CLIQUE_LONGO (proposta: 0,75 s) e se o preparo vencido cancela sem punição.
- **RunManager e TimeBar:** manter dois scripts ou juntar a barra de tempo no RunManager (seção 14).